

Идентифицирующие материалы программы СМК-НТТС

Соловьев В. В., i@viktor-solovev.ru

Санкт-Петербургский государственный архитектурно-строительный университет, Санкт-Петербург

SPIN-код: 6187-7512, ORCID: 0009-0003-5573-8862

Для практической верификации разработанной математической модели и алгоритмов многозадачной координации реализована программа «Симулятор многозадачной координации групп роботизированных наземных транспортно - технологических средств» (СМК-НТТС) на языке программирования Go¹. Полный текст программного средства представлен в приложениях².

Выбор данного языка обусловлен рядом его фундаментальных преимуществ, критически важных для построения надёжных, производительных и масштабируемых систем управления роботизированными группами. Одновременно с этим, автор имеет большой опыт³ в написании распределённых и эффективных программных средств и систем на этом языке программирования высокого уровня. Кроме озвученных фактов, имеется еще ряд безусловных преимуществ.

Так, Go изначально спроектирован для создания высоконагруженных сетевых сервисов и распределенных систем. Имеет встроенную поддержку конкурентности через механизм горутин (goroutines) и каналов (channels), что позволяет просто и эффективно моделировать поведение множества автономных НТТС, действующих параллельно и взаимодействующих друг с другом. Это идеально соответствует задаче моделирования децентрализованной координации, где каждая машина принимает решения независимо, но с учетом состояния группы.

Во-вторых, язык обеспечивает исключительную производительность, близкую к C/C++, при этом сохраняя простоту синтаксиса и скорость разработки, характерную для интерпретируемых языков. Это позволит реализовать вычислительно интенсивные симуляции с большим количеством итераций и машин за приемлемое время, что является ключевым требованием для проведения серийных экспериментов и получения статистически значимых результатов.

¹ исходный код и описание размещено на <https://go.dev>

² исходный код для самостоятельного проведения эксперимента или предложений для апробации доступен на <https://github.com/dreddsa5dies/phd>

³ программы с открытым исходным кодом размещены на <https://github.com/dreddsa5dies>, коммерческие продукты разрабатывались в компаниях Qtch и YADRO

В-третьих, Go обладает превосходной поддержкой кроссплатформенной компиляции. Это означает, что разработанная симуляционная среда может быть собрана и запущена без изменений на различных операционных системах, что значительно упрощает ее распространение и использование, а также потенциальную интеграцию с различными аппаратными платформами.

Наконец, статическая типизация и строгая система компиляции Go способствуют созданию более надежного и предсказуемого кода. Это критически важно для научных исследований, где воспроизводимость результатов является основополагающим принципом. Компилятор выявляет множество потенциальных ошибок на этапе сборки, что снижает вероятность возникновения трудноуловимых багов в процессе выполнения симуляций.

Наиболее существенным недостатком использования этого языка программирования в части, касающейся применения в НТТС, является работа планировщика языка Go и использования механизма «остановки» для сборки неиспользуемых объектов в памяти ОС. Это значит, что его нельзя применять в процессах, связанных с непосредственной опасностью для людей и окружающего мира. Для этого стоит применять только ОС реального времени и языки программирования без временных пауз. Но это не относится к теме исследования и применение Go целесообразно для проведения эксперимента.

Общая архитектура реализации определена принципами модульности и расширяемости. Система состоит из нескольких компонентов представленных на рисунке 1.

Модуль стратегии (прил. А.8) координации реализуют интерфейс Strategy (лист. 1), что позволяет легко добавлять новые алгоритмы распределения задач и сравнивать их в единой среде. Каждая стратегия должна удовлетворять требованиям этого интерфейса, т. е. запускаться, принимая набор машин и задач, и возвращать метрики для последующего анализа. Метод String() предназначен для возврата наименования стратегии.

Листинг 1: Интерфейс стратегий по выбору задач

```
5 | type Strategy interface {  
    |     Run(machines []models.Machine, tasks []models.Task) StepMetrics  
    |     String() string  
    | }
```

В моделях данных определяются структуры данных для представления машин (прил. А.4) и задач (прил. А.5), включая их характеристики. Пример описания модели машин с распределением по типам представлен в описаниях 2, 3. Описаны такие типы, как экскаватор с набором оборудования: ковш,

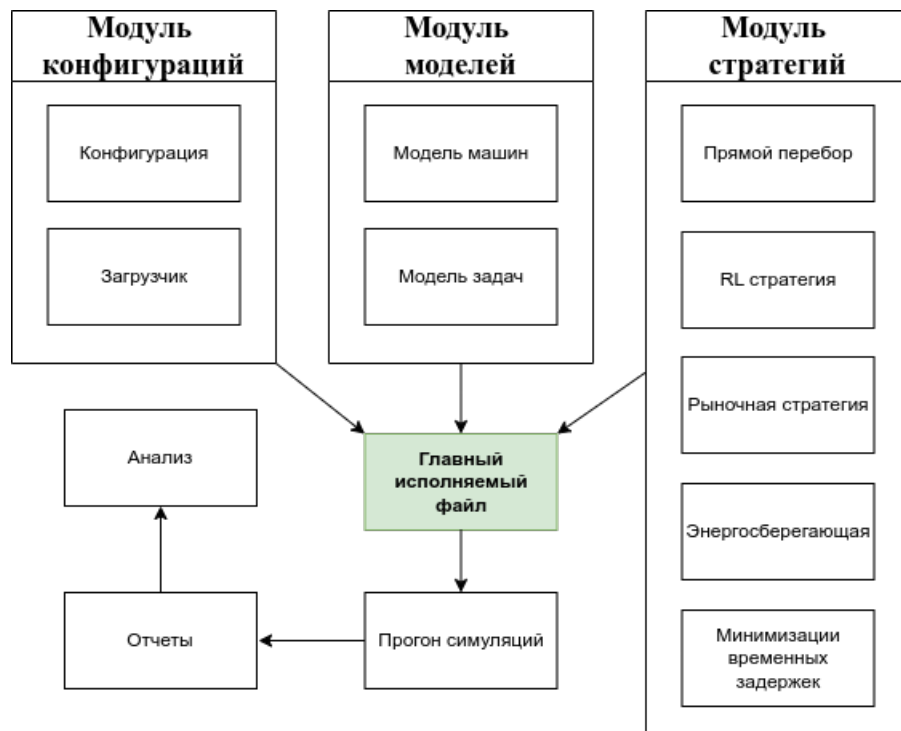


Рис. 1: Модульная структура программной реализации метода автоматизированной многозадачной координации.

грейфер и рыхлитель (типы 1, 3, 9), бульдозер с отвалом и рыхлителем (типы 2, 9), погрузчик с ковшом, захватом и вилами (типы 1, 7, 14) и другие.

Листинг 2: Модель данных машин

```

// Machine описывает машину
type Machine struct {
    // уникальный идентификатор
    ID string 'json:"id" yaml:"id"'
    // тип оборудования
    Equipment []TypeEquipment 'json:"equipment" yaml:"equipment"'
    // запас энергии
    Energy float64 'json:"energy" yaml:"energy"'
    // выполняемая задача
    AssignedTask *Task
}

```

Симуляционное ядро (прил. А.6) отвечает за инициализацию эксперимента, последовательный запуск выбранной стратегии на заданном количестве шагов и сбор метрик производительности.

Утилиты и конфигурация (прил. А.2 и А.3) обеспечивают загрузку параметров эксперимента из внешнего файла типа YAML⁴ и вспомогательные

⁴ формат описания данных, близкий к языкам разметки с поддержкой комментариев

Листинг 3: Описание типов оборудования

```
const (  
    Bucket      TypeEquipment = iota + 1 // ковш  
    Dump        // отвал  
    Grapple     // грейфер  
    Brushes     // щетки  
    Cutter     // фреза  
    Plow        // плуг  
    Catch       // захват  
    Tamp        // трамбовочное устройство  
    Ripper      // рыхлитель  
    Scraper     // скребок  
    Conveyor    // конвейер ленточный  
    Knife       // ножотвал- автогрейдера  
    VCompactor  // виброплита  
    Pitchfork   // вилы для( паллет)  
    Thrower     // метатель  
)
```

функции.

Аналитический модуль (прил. А.14) позволяет генерировать сводные отчеты в формате CSV⁵ и формирует результаты в виде графиков для наглядного сравнения эффективности различных стратегий.

Таким образом, реализация на языке Go предоставляет надежную, производительную и гибкую платформу для проведения вычислительных экспериментов, что позволяет всесторонне оценить эффективность предложенного метода автоматизированной многозадачной координации по ключевым метрикам: времени выполнения задач, энергопотреблению и устойчивости к сбоям связи, а также провести его сравнительный анализ с существующими аналогами.

А Программный код

А.1 Главный исполнительный файл

Листинг 1: файл strategies/main.go

```
package main  
  
import (  
    "fmt"
```

⁵ простой табличный формат описания данных

```

5  "log"
   "math/rand"
   "time"

   "github.com/dreddsa5dies/phd/Dissertation/code/src/engine"
10  "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
   "github.com/dreddsa5dies/phd/Dissertation/code/src/strategies"
   "github.com/dreddsa5dies/phd/Dissertation/code/src/utils"
)

15 func main() {
   // чтение конфигурационного файла
   config, err := utils.LoadConfig("config/config.yaml")
   if err != nil {
20     log.Fatal(err)
   }

   // инициализация стратегий поведения
   allStrategies := []strategies.Strategy{
25     // Стратегия "прямого перебора"
     &strategies.DirectSearch{},
     // Рыночная стратегия
     &strategies.MarketStrategy{},
     // "RL"
30     &strategies.RLStrategy{
        Q:      make(map[string]map[int]float64),
        LR:      0.5,
        Gamma:   0.9,
        Epsilon: 0.2,
        MinEps:  0.01,
35     Decay:   0.995,
    },
     // Энергосберегающая стратегия
     &strategies.EnergyEfficientStrategy{},
     // Стратегия минимизации временных задержек
40     &strategies.DelayMinimizationStrategy{
        // коэффициенты для
        Alpha: 0.5, //nolint:mnd
        Beta:  1.0,
    },
45 }

   // инициализация группы машин
   machines := make([]models.Machine, len(config.Machines))
   for i, rc := range config.Machines {
50     machines[i] = models.Machine{
        ID:      rc.ID,
        Equipment: rc.Equipment,
    }
   }

55 // инициализация задач

```

```

tasks := make([]models.Task, len(config.Tasks))
tNow := time.Now().Unix()
for i, tc := range config.Tasks {
60     tasks[i] = models.Task{
        ID:          tc.ID,
        RequiredEquipment: tc.RequiredEquipment,
        EnergyCost:    tc.EnergyCost,
65         CreatedAt:    tNow,
    }

    // обеспечение временной последовательности
    tNow++
70 }

// добавление задач в массив
// наблюдаемых машинами случайный( процесс)
tasks = tasksFromMachines(
    config.Simulation.PoissonLambda,
75     tasks,
    config.Simulation.Steps,
)

// перемешивание задач в массиве случайным образом
80 r := rand.New(rand.NewSource(time.Now().UnixNano()))
r.Shuffle(len(tasks), func(i, j int) {
    tasks[i], tasks[j] = tasks[j], tasks[i]
})

85 // прогон всех стратегий на одинаковых исходных данных
// каждая стратегия прогоняется заданное количество шагов
// формируется отчет
results := make(map[string]map[string]strategies.Metrics)

90 // Вычисляем суммарную энергию всех задач
var totalTaskEnergy float64
for _, task := range tasks {
    totalTaskEnergy += task.EnergyCost
95 }

// Определяем три сценария по энергии машин
scenarios := map[string]float64{
    // СУМэнергия[ всех HTTC] = СУМэнергия[ всех задач] / 2 -> min
    "min": totalTaskEnergy / 2.0,
100    // СУМэнергия[ всех HTTC] = СУМэнергия[ всех задач] -> equal
    "equal": totalTaskEnergy,
    // СУМэнергия[ всех HTTC] = СУМэнергия[ всех задач] * 2 -> max
    "max": totalTaskEnergy * 2.0,
}

105 // Восстанавливаем исходные машины до( модификации энергии)
originalMachinesConfig := config.Machines

```

```

110 for scenarioName, totalMachineEnergy := range scenarios {
    fmt.Printf("\n+++ Сценарий: %s суммарная( энергия машин = %.2f)\n",
        scenarioName, totalMachineEnergy)

    // Создаём новые машины с той же структурой, но общей энергией,
    // равной сценарию
    machines := make([]models.Machine, len(originalMachinesConfig))

115 // Распределяем totalMachineEnergy равномерно между машинами
    energyPerMachine := totalMachineEnergy / float64(len(machines))
    for i, rc := range originalMachinesConfig {
        machines[i] = models.Machine{
            ID:      rc.ID,
120         Equipment: rc.Equipment,
            Energy:   energyPerMachine, // равномерное распределение
        }
    }

125 // Инициализация подотчёта- для этого сценария
    results[scenarioName] = make(map[string]strategies.Metrics)

    // Запуск всех стратегий в этом сценарии
    for _, s := range allStrategies {
130         strategyName := fmt.Sprintf("%s", s)
        fmt.Printf("Старт эксперимента со стратегией: %s\n", strategyName)
        engine.RunSimulation(
            machines,
            tasks,
135             s,
            config.Simulation.Steps,
            results[scenarioName], // передаём подотчёт-
        )
    }
140 }

    // сохранение результатов
    err = engine.GenerateReport(results)
    if err != nil {
145         fmt.Printf("Ошибка сохранения отчета: %s\n", err)
        return
    }

    fmt.Println("Эксперименты закончены")
150 }

// случайный процесс добавления новых задач, наблюдаемых машинами
// один для ВСЕХ стратегий
func tasksFromMachines(lambda float64, tasks []models.Task, steps int)
155 []models.Task {
    r := rand.New(rand.NewSource(time.Now().UnixNano()))

    tNow := time.Now().Unix()

```

```

160   for range steps {
      if r.Float64() < lambda {
          // всего 15 типов в этой симуляции
          eq := 1 + r.Intn(14) //nolint:mnd
          // определим энергоемкость задачи: максимум не больше 3 * 1000
          en := 1 + r.Intn(2999) //nolint:mnd

165         newTask := models.Task{
            ID:                fmt.Sprintf("T%d", len(tasks)+1),
            RequiredEquipment: models.TypeEquipment(eq),
            EnergyCost:         float64(en),
            CreatedAt:          tNow,
170         }
         tasks = append(tasks, newTask)

          // обеспечение временной последовательности
          tNow++
175     }
    }

    return tasks
}

```


A.2 Конфигурационный файл

Листинг 2: файл config.yaml

```
simulation:
  steps: 5
  poisson_lambda: 0.5
5 machines:
  # экскаваторы
  - id: M1
    equipment:
 10     - 1
      - 3
      - 9
  # бульдозеры
  - id: M2
    equipment:
 15     - 2
      - 9
  # погрузчики
  - id: M3
    equipment:
 20     - 1
      - 7
      - 14
  # снегоуборочные машины
  - id: M4
 25     equipment:
        - 4
        - 5
        - 6
        - 15
 30 # автогрейдеры
  - id: M5
    equipment:
      - 11
  # виброплиты
 35 - id: M6
    equipment:
      - 8
      - 13
  # конвейеры
 40 - id: M7
    equipment:
      - 10
  # скреперы
  - id: M8
 45     equipment:
        - 12
tasks:
```

50	- id: T1
	required_equipment: 1
	energy_cost: 1500
	- id: T2
	required_equipment: 2
	energy_cost: 10
55	- id: T3
	required_equipment: 3
	energy_cost: 1500
	- id: T4
	required_equipment: 4
60	energy_cost: 10
	- id: T5
	required_equipment: 5
	energy_cost: 1500
	- id: T6
65	required_equipment: 6
	energy_cost: 10
	- id: T7
	required_equipment: 7
	energy_cost: 1500
70	- id: T8
	required_equipment: 8
	energy_cost: 10
	- id: T9
	required_equipment: 9
75	energy_cost: 1500
	- id: T10
	required_equipment: 10
	energy_cost: 10
	- id: T11
80	required_equipment: 11
	energy_cost: 1500
	- id: T12
	required_equipment: 12
	energy_cost: 10
85	- id: T13
	required_equipment: 13
	energy_cost: 1500
	- id: T14
	required_equipment: 14
90	energy_cost: 10
	- id: T15
	required_equipment: 15
	energy_cost: 1500
	- id: T16
95	required_equipment: 1
	energy_cost: 10
	- id: T17
	required_equipment: 2
	energy_cost: 1500
100	- id: T18

	required_equipment: 3
	energy_cost: 10
	- id: T19
105	required_equipment: 4
	energy_cost: 1500
	- id: T20
	required_equipment: 5
	energy_cost: 10
110	- id: T21
	required_equipment: 6
	energy_cost: 1500
	- id: T22
	required_equipment: 7
	energy_cost: 10
115	- id: T23
	required_equipment: 8
	energy_cost: 1500
	- id: T24
120	required_equipment: 9
	energy_cost: 10
	- id: T25
	required_equipment: 10
	energy_cost: 1500
	- id: T26
125	required_equipment: 11
	energy_cost: 10
	- id: T27
	required_equipment: 12
	energy_cost: 1500
130	- id: T28
	required_equipment: 13
	energy_cost: 10
	- id: T29
135	required_equipment: 14
	energy_cost: 1500
	- id: T30
	required_equipment: 15
	energy_cost: 10

A.3 Утилита загрузки конфигурации

Листинг 3: файл loader.go

```
package utils

import (
    "os"
    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
    "gopkg.in/yaml.v2"
)

5

10 type SimulationConfig struct {
    Steps          int          'yaml:"steps"'
    PoissonLambda float64    'yaml:"poisson_lambda"'
}

15 type Config struct {
    Machines []models.Machine 'yaml:"machines"'
    Tasks    []models.Task      'yaml:"tasks"'
    Simulation SimulationConfig 'yaml:"simulation"'
}

20 // LoadConfig инициализирует конфигурацию из файла
func LoadConfig(path string) (*Config, error) {
    data, err := os.ReadFile(path)
    if err != nil {
25         return nil, err
    }

    var config Config
    err = yaml.Unmarshal(data, &config)
30 if err != nil {
        return nil, err
    }

    return &config, nil
35 }
```

A.4 Описание модели машины

Листинг 4: файл robot.go

```
package models

// Machine описывает машину
type Machine struct {
5  // уникальный идентификатор
  ID string `json:"id" yaml:"id"`
  // тип оборудования
  Equipment []TypeEquipment `json:"equipment" yaml:"equipment"`
  // запас энергии
10  Energy float64 `json:"energy" yaml:"energy"`
  // выполняемая задача
  AssignedTask *Task
}

15 // TypeEquipment - типы оборудования
// Примеры по технике:
//
// экскаватор : ковш, грейфер, рыхлитель (1, 3, 9)
// бульдозер : отвал, рыхлитель (2, 9)
20 // погрузчик : ковш, захват, вилы для( паллет) (1, 7, 14)
// снегоуборочная машина : плуг, фреза, щётка, метатель (6, 5, 4, 15)
// автогрейдер : ножотвал- (11)
// виброплита : трамбовочное устройство, виброплита (8, 13)
// конвейер : конвейер ленточный (10)
25 // скрепер : скребок (12)
type TypeEquipment uint

const (
  Bucket      TypeEquipment = iota + 1 // ковш
30  Dump                               // отвал
  Grapple                               // грейфер
  Brushes                               // щетки
  Cutter                               // фреза
  Plow                               // плуг
35  Catch                               // захват
  Tamp                               // трамбовочное устройство
  Ripper                               // рыхлитель
  Scraper                               // скребок
  Conveyor                               // конвейер ленточный
40  Knife                               // ножотвал- автогрейдера
  VCompactor                               // виброплита
  Pitchfork                               // вилы для( паллет)
  Thrower                               // метатель
)

45 func IntersectTypeEquipment(in TypeEquipment, t []TypeEquipment) bool {
  for i := range t {
    if t[i] == in {
```

```
        return true
50    }
    }
    return false
}

55 func TotalEnergyMachines(in []Machine) float64 {
    total := 0
    for i := range in {
        total += int(in[i].Energy)
    }
60    return float64(total)
}
```

A.5 Описание модели задачи

Листинг 5: файл task.go

```
package models

// Task описывает задачу
type Task struct {
5  // уникальный идентификатор
  ID string `json:"id" yaml:"id"`
  // требуемый тип оборудования
  RequiredEquipment TypeEquipment `json:"required_equipment" yaml:"
    required_equipment"`
  // требуемые затраты по энергии
10  EnergyCost float64 `json:"energy_cost" yaml:"energy_cost"`
  // время создания в UnixTime
  CreatedAt int64
}

15 // удаление задачи из массива
func RemoveTask(tasks []Task, task Task) []Task {
  for i, t := range tasks {
    if t.ID == task.ID {
      return append(tasks[:i], tasks[i+1:]...)
20    }
  }
  return tasks
}

25 func TotalEnergyTasks(in []Task) float64 {
  total := 0
  for i := range in {
    total += int(in[i].EnergyCost)
  }
30  return float64(total)
}
```

A.6 Исполнение модели

Листинг 6: файл simulation.go

```
package engine

import (
    "time"
    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
    "github.com/dreddsa5dies/phd/Dissertation/code/src/strategies"
)

// Прогон экспериментов
func RunSimulation(machines []models.Machine, tasks []models.Task,
    strategy strategies.Strategy, steps int, report map[string]strategies
    .Metrics) {
    // подготовка отчета
    tNow := time.Now()
    metrics := strategies.Metrics{
        TotalTasks:      len(tasks),
        TotalMachine:     len(machines),
        TotalEnergyMachines: models.TotalEnergyMachines(machines),
        AllMachines:      machines,
        TotalEnergyTasks: models.TotalEnergyTasks(tasks),
        AllTasks:         tasks,
        TotalTime:        "",
    }

    // прогон стратегии заданное количество шагов
    runMetrics := make([]strategies.StepMetrics, steps)
    for i := range steps {
        // Копируем список задач и машин
        // независимые прогоны стратегий
        taskList := make([]models.Task, len(tasks))
        copy(taskList, tasks)

        machineList := make([]models.Machine, len(machines))
        copy(machineList, machines)

        runMetrics[i] = strategy.Run(machineList, taskList)
        runMetrics[i].Step = i + 1
    }

    metrics.StepMetric = runMetrics
    // фиксация итогового времени
    metrics.TotalTime = time.Since(tNow).String()

    report[strategy.String()] = metrics
}
```


A.7 Подготовка отчета

Листинг 7: файл reporter.go

```
package engine

import (
    "encoding/json"
    "os"
)

// GenerateReport записывает отчет о прогоне стратегии в файл формата
// json
func GenerateReport(results any) error {
    file, err := os.Create("report.json")
    if err != nil {
        return err
    }
    defer file.Close()

    // Используем json.Encoder для записи в файл
    encoder := json.NewEncoder(file)
    encoder.SetIndent("", " ") // Форматирование с отступами
    опционально()

    err = encoder.Encode(results)
    if err != nil {
        return err
    }

    return nil
}
```

A.8 Общий интерфейс запуска стратегий

Листинг 8: файл strategy.go

```
package strategies

import (
    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
5 )

// Метрики
type Metrics struct {
    // Общее количество задач всего
    10 TotalTasks int 'json:"totalTasks"'
    // Общее количество машин всего
    TotalMachine int 'json:"totalMachine"'
    // Общая энергоемкость машин
    TotalEnergyMachines float64 'json:"totalEnergyMachines"'
    15 // Список машин
    AllMachines []models.Machine 'json:"allMachines"'
    // Общее энергоемкость задач
    TotalEnergyTasks float64 'json:"totalEnergyTasks"'
    // Список задач
    20 AllTasks []models.Task 'json:"allTasks"'
    // Общее время выполнения
    TotalTime string 'json:"totalTime"'
    // Метрики каждого шага
    StepMetric []StepMetrics 'json:"stepMetrics"'
    25 }

type StepMetrics struct {
    Step int 'json:"step"' // Номер прогона
    // Количество выполненных задач
    30 LenTasksDone int 'json:"lenTasksDone"'
    // Перечень исполненных задач по ID
    TasksDone []string 'json:"tasksDone"'
    // Энергозатраты машин
    EnergyUsed float64 'json:"energyUsed"'
    35 // Не исполненные задачи
    LenNotExecTasks int 'json:"lenNotExecTasks"'
    // Реальное время выполнения шага
    RealTime string 'json:"realTime"'
    40 }

// общий интерфейс стратегий по выбору задач
type Strategy interface {
    Run(machines []models.Machine, tasks []models.Task) StepMetrics
    String() string
    45 }
```

A.9 Стратегия минимизации временных задержек

Листинг 9: файл delay.go

```
package strategies

import (
    "math"
    "time"

    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
)

// DelayMinimizationStrategy реализует стратегию минимизации временных
// задержек
type DelayMinimizationStrategy struct {
    Alpha float64 // вес координации
    Beta  float64 // вес: задержка * сложность
}

func (s *DelayMinimizationStrategy) String() string {
    return "Минимизация ВрЗ"
}

func (s *DelayMinimizationStrategy) Run(machines []models.Machine, tasks
    []models.Task) StepMetrics {
    start := time.Now()

    // Соберём начальную энергию до( прогонки)
    sumEnergy := func() float64 {
        var sum float64
        for i := range machines {
            if machines[i].Energy > 0 {
                sum += machines[i].Energy
            }
        }
        return sum
    }
    initialTotalEnergy := sumEnergy()

    // Вспом. функции
    countUnfinished := func() int {
        var n int
        for i := range tasks {
            if tasks[i].EnergyCost > 0 {
                n++
            }
        }
        return n
    }

    computeW := func(t *models.Task) float64 {
        age := time.Now().Unix() - t.CreatedAt
```

```

    if age < 0 {
        age = 0
    }
50   return float64(age)
}

totalMachines := len(machines)
maxIters := 10_000 + 10*len(tasks)*maximum(1, totalMachines)
55

// prevChoices для оценки Pj частотная( оценка предыдущей итерации)
prevChoices := make(map[string]int)

// Набор завершённых задач за весь прогон
60 doneSet := make(map[string]struct{})
var doneList []string

// Внутренний итерационный цикл стратегии – выполняем до сходимости,
// но в отчёт запишем только итог за весь прогон.
65 for iter := 0; iter < maxIters; iter++ {
    // Стопкритерии-
    if countUnfinished() == 0 || sumEnergy() == 0 {
        break
    }
70

    // --- ФАЗА ВЫБОРА последовательно(, детерминированно) ---
    // Для каждой машины выбираем индекс задачи или -1
    choices := make([]int, len(machines))
    for i := range choices {
75         choices[i] = -1
    }

    for i := range machines {
        m := &machines[i]
80         if m.Energy <= 0 {
            continue
        }

        bestIdx := -1
85         bestVal := math.Inf(1)

        for j := range tasks {
            t := &tasks[j]
            if t.EnergyCost <= 0 {
90                 continue
            }

            if !models.IntersectTypeEquipment(t.RequiredEquipment, m.
Equipment) {
                continue
            }

95             Wj := computeW(t)
            Qj := t.EnergyCost
            var Pj float64

```

```

100         if totalMachines > 0 {
            Pj = float64(prevChoices[t.ID]) / float64(totalMachines)
        }
        val := s.Beta*Wj*Qj - s.Alpha*Pj
        if val < bestVal {
            bestVal = val
            bestIdx = j
105     }
    }
    choices[i] = bestIdx
}

110 // --- Агрегация выборов ---
// taskIdx -> []machineIdx
chosenMap := make(map[int][]int)
currChoices := make(map[string]int)
for mi, tIdx := range choices {
115     if tIdx >= 0 {
        chosenMap[tIdx] = append(chosenMap[tIdx], mi)
        currChoices[tasks[tIdx].ID]++
    }
}

120 // --- ФАЗА ИСПОЛНЕНИЯ: только машины, выбравшие задачу, вносят
вклад ---
var energySpentThisIter float64
for tIdx := range tasks {
    t := &tasks[tIdx]
125     if t.EnergyCost <= 0 {
        continue
    }
    mList := chosenMap[tIdx]
    if len(mList) == 0 {
130         continue
    }

    initialCost := t.EnergyCost
    var totalContrib float64

135     // Машины последовательным образом вносят вклад чтобы( избежать
гонок)
    for _, mi := range mList {
        if mi < 0 || mi >= len(machines) {
            continue
140        }
        m := &machines[mi]
        if m.Energy <= 0 {
            continue
        }

145        delta := m.Energy
        remaining := initialCost - totalContrib

```

```

    if delta > remaining {
        delta = remaining
150    }
    // уменьшаем энергию машины и добавляем вклад
    m.Energy -= delta
    totalContrib += delta
    if totalContrib >= initialCost {
155        break
    }
}

energySpentThisIter += totalContrib

160
if totalContrib >= initialCost {
    // задача завершена
    t.EnergyCost = 0
    if _, ok := doneSet[t.ID]; !ok {
165        doneSet[t.ID] = struct{}{}
        doneList = append(doneList, t.ID)
    }
} else if totalContrib > 0 {
    // частично выполнена
170    t.EnergyCost = initialCost - totalContrib
}
}

// Обновляем prevChoices для следующей итерации
175 prevChoices = make(map[string]int)
for k, v := range currChoices {
    prevChoices[k] = v
}

180 // Если прогресса нет – завершаем
if energySpentThisIter == 0 {
    break
}
}

185
// Подсчёт итоговых метрик за весь прогон один( StepMetrics)
finalTotalEnergy := sumEnergy()
energyUsed := initialTotalEnergy - finalTotalEnergy

190 stepMetrics := StepMetrics{
    EnergyUsed:    energyUsed,
    TasksDone:     doneList,
    LenTasksDone:  len(doneList),
    LenNotExecTasks: 0, // заполним ниже
195    RealTime:     time.Since(start).String(),
}

for i := range tasks {
    if tasks[i].EnergyCost > 0 {

```

```

200|         stepMetrics.LenNotExecTasks++
        }
    }

    return stepMetrics
205 }

// maximum – вспомогательная функция вычисления максимума
func maximum(a, b int) int {
    if a >= b {
210         return a
    }
    return b
}

```

A.10 Энергосберегающая стратегия

Листинг 10: файл energy.go

```
package strategies

import (
    "math"
    "time"

    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
)

// энергосберегающая стратегия
type EnergyEfficientStrategy struct{}

// String возвращает имя стратегии
func (s *EnergyEfficientStrategy) String() string {
    return "Энергосберегающая"
}

func (s *EnergyEfficientStrategy) Run(machines []models.Machine, tasks
    []models.Task) StepMetrics {
    start := time.Now()

    // Подсчёт начальной энергии
    sumEnergy := func() float64 {
        var sum float64
        for i := range machines {
            if machines[i].Energy > 0 {
                sum += machines[i].Energy
            }
        }
        return sum
    }
    initialEnergy := sumEnergy()

    // Набор выполненных задач
    doneSet := make(map[string]struct{})
    var doneList []string

    // Внутренний цикл выполнения до завершения или исчерпания энергии
    maxIters := 10_000 + 10*len(tasks)*maximum(1, len(machines))
    for iter := 0; iter < maxIters; iter++ {
        if sumEnergy() == 0 {
            break
        }

        progress := false

        // --- ФАЗА ВЫБОРА + ИСПОЛНЕНИЯ ---
        for i := range machines {
```



```

50     m := &machines[i]
    if m.Energy <= 0 {
        continue
    }

    // выбор доступной задачи с минимальным  $Q_j/R_i$ 
    bestIdx := -1
55     bestVal := math.Inf(1)
    for j := range tasks {
        t := &tasks[j]
        if t.EnergyCost <= 0 {
            continue
60         }
        if !models.IntersectTypeEquipment(t.RequiredEquipment, m.
Equipment) {
            continue
        }

65         val := t.EnergyCost / m.Energy
        if val < bestVal {
            bestVal = val
            bestIdx = j
        }
70     }

    if bestIdx == -1 {
        continue
    }

75     // --- Исполнение выбранной задачи ---
    t := &tasks[bestIdx]
    initialCost := t.EnergyCost
    delta := m.Energy
80     if delta > initialCost {
        delta = initialCost
    }
    m.Energy -= delta
    t.EnergyCost -= delta
85     progress = true

    if t.EnergyCost <= 0 {
        t.EnergyCost = 0
        if _, ok := doneSet[t.ID]; !ok {
90             doneSet[t.ID] = struct{}{}
            doneList = append(doneList, t.ID)
        }
    }
}

95 if !progress {
    break
}

```

```

100     }

    // Итоговые метрики
    finalEnergy := sumEnergy()
    energyUsed := initialEnergy - finalEnergy

105     stepMetrics := StepMetrics{
        EnergyUsed:    energyUsed,
        TasksDone:     doneList,
        LenTasksDone:  len(doneList),
110        LenNotExecTasks: 0,
        RealTime:      time.Since(start).String(),
    }

    for i := range tasks {
115        if tasks[i].EnergyCost > 0 {
            stepMetrics.LenNotExecTasks++
        }
    }

    // возврат метрик
120    return stepMetrics
}

```

A.11 Стратегия рыночных методов

Листинг 11: файл market.go

```
package strategies

import (
    "sort"
    "time"

    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
)

// MarketStrategy – реализация простого рынка / контрактнет- (inspired
// by Dias 2006)
type MarketStrategy struct{}

func (s *MarketStrategy) String() string {
    return "Рыночных методов"
}

// простой "cost" для ставки: чем меньше – лучше
// cost = (remainingTaskEnergy) / (availableMachineEnergy + eps)
// то есть машина с большим запасом энергии даёт более дешёвую"" ставку
func bidCost(taskEnergy float64, machineEnergy float64) float64 {
    eps := 1e-6
    return taskEnergy / (machineEnergy + eps)
}

// Run проводит один полный прогон стратегии; в отчёт записывает
// итоговые метрики
func (s *MarketStrategy) Run(machines []models.Machine, tasks []models.
    Task) StepMetrics {
    start := time.Now()

    // начальная сумма энергии
    sumEnergy := func() float64 {
        var sum float64
        for i := range machines {
            if machines[i].Energy > 0 {
                sum += machines[i].Energy
            }
        }
        return sum
    }
    initialEnergy := sumEnergy()

    doneSet := make(map[string]struct{})
    var doneList []string

    // Простая реализация аукционов: для каждой задачи запрашиваем ставки
    у всех подходящих машин.
```

```

45 // Затем выбираем победителей(). Для крупной задачи можно
    // распределить между несколькими машинами пропорционально их energy.
    // Проходим задачами детерминированно по( индексу) – это даёт
    // воспроизводимость.
    for ti := range tasks {
        t := &tasks[ti]
        if t.EnergyCost <= 0 {
50             continue
        }

        // собрать кандидатов
        type bid struct {
55             machineIdx int
                cost      float64
        }
        bids := make([]bid, 0, len(machines))
        for mi := range machines {
60             m := &machines[mi]
                if m.Energy <= 0 {
                    continue
                }
                if !models.IntersectTypeEquipment(t.RequiredEquipment, m.Equipment
) {
65                     continue
                }
                c := bidCost(t.EnergyCost, m.Energy)
                bids = append(bids, bid{machineIdx: mi, cost: c})
        }

70         if len(bids) == 0 {
            // никто не может выполнить задачу
            continue
        }

75         // сортируем по возрастанию стоимости меньше( cost – лучше)
        sort.Slice(bids, func(i, j int) bool {
            return bids[i].cost < bids[j].cost
        })

80         // Если самая дешевая ставка способна завершить задачу – отдаем ей
        // задачу полностью.
        // Иначе распределяем между топ-K машин пропорционально их энергии.
        top := bids[0]
        mTop := &machines[top.machineIdx]
85         if mTop.Energy >= t.EnergyCost {
            // простое присвоение
            delta := t.EnergyCost
            if delta > mTop.Energy {
                delta = mTop.Energy
90            }
            mTop.Energy -= delta
            t.EnergyCost -= delta

```

```

    if t.EnergyCost <= 0 {
        t.EnergyCost = 0
105     if _, ok := doneSet[t.ID]; !ok {
            doneSet[t.ID] = struct{}{}
            doneList = append(doneList, t.ID)
        }
    }
100     continue
}

// Иначе распределяем по нескольким машинам: берем топ-N включая(
тех, у кого >0 energy)
// Берём пока не покроем задачу или победителей не останется.
105 remaining := t.EnergyCost
// собираем список подходящих машин в порядке предпочтения
preferred := make([]int, 0, len(bids))
for _, b := range bids {
    preferred = append(preferred, b.machineIdx)
110 }

// простая схема: пропорционально энергии (bounded)
totalAvail := 0.0
for _, mi := range preferred {
115     totalAvail += machines[mi].Energy
}
if totalAvail <= 0 {
    // нет доступной энергии
    continue
120 }

for _, mi := range preferred {
    if remaining <= 0 {
        break
125     }
    m := &machines[mi]
    if m.Energy <= 0 {
        continue
    }
130     // доля этой машины
    share := (m.Energy / totalAvail) * t.EnergyCost
    // ограничим по оставшейся энергии машины и по оставшейся задаче
    if share > m.Energy {
        share = m.Energy
135     }
    if share > remaining {
        share = remaining
    }
    m.Energy -= share
140     remaining -= share
}
// уменьшение задачи на внесённый вклад
done := (t.EnergyCost - remaining)

```

```

145     if done > 0 {
        if remaining <= 0 {
            t.EnergyCost = 0
            if _, ok := doneSet[t.ID]; !ok {
                doneSet[t.ID] = struct{}{}
                doneList = append(doneList, t.ID)
150         }
        } else {
            t.EnergyCost = remaining
        }
    }
155 }

finalEnergy := sumEnergy()
energyUsed := initialEnergy - finalEnergy

160 stepMetrics := StepMetrics{
    EnergyUsed:    energyUsed,
    TasksDone:     doneList,
    LenTasksDone:  len(doneList),
    LenNotExecTasks: 0,
165    RealTime:     time.Since(start).String(),
}

for i := range tasks {
    if tasks[i].EnergyCost > 0 {
        stepMetrics.LenNotExecTasks++
170    }
}

return stepMetrics
}

```

А.12 Стратегия RL-методов

Листинг 12: файл rl.go

```
package strategies

import (
    "math"
    "math/rand"
    "sync"
    "time"

    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
)

// RLStrategy – стратегия на базе Q-learning модифицированная( для
// задач распределения)
type RLStrategy struct {
    // Q: map[machineID]map[equipmentType]Qvalue
    Q map[string]map[int]float64

    // гиперпараметры обучения
    LR      float64 // learning rate
    Gamma   float64 // discount factor
    Epsilon float64 // начальное epsilon для epsilon-greedy
    MinEps   float64 // минимальное epsilon
    Decay    float64 // decay per Run call

    mu sync.Mutex // защита Q если стратегия используется параллельно
}

func (s *RLStrategy) String() string {
    return "RL (Q-learning)"
}

// Run выполняет один шаг целый( прогон стратегии).
// Реализует интерактивный Q-learning: машины действуют, получают
// награды, обновляют Q.
func (s *RLStrategy) Run(machines []models.Machine, tasks []models.Task)
    StepMetrics {
        start := time.Now()
        rng := rand.New(rand.NewSource(time.Now().UnixNano()))

        // вспомогательные функции
        sumEnergy := func() float64 {
            var sum float64
            for i := range machines {
                if machines[i].Energy > 0 {
                    sum += machines[i].Energy
                }
            }
            return sum
        }
    }
```

```

}
countUnfinished := func() int {
    var n int
    for i := range tasks {
50         if tasks[i].EnergyCost > 0 {
            n++
        }
    }
    return n
55 }

// Подготовка итоговых метрик
doneSet := make(map[string]struct{})
var doneList []string

60
initialTotalEnergy := sumEnergy()

// Параметры итераций внутри прогона
maxIters := 10000 + 10*len(tasks)*maximum(1, len(machines))
65

// Основной итерационный цикл: машины действуют и учатся
for iter := 0; iter < maxIters; iter++ {
    // стопкритерии-
    if sumEnergy() == 0 || countUnfinished() == 0 {
70         break
    }

    anyProgress := false

75
    // для каждой машины – выбор задачи и исполнение
    for mi := range machines {
        m := &machines[mi]
        if m.Energy <= 0 {
            continue
80        }

        // Соберём доступные действия (equipment types задач, которые
        машина может делать)
        availableActions := make([]int, 0, 8)
        taskIdxByAction := make(map[int][]int) // action -> список
        индексов задач этого типа
85        for ti := range tasks {
            t := &tasks[ti]
            if t.EnergyCost <= 0 {
                continue
            }
            if !models.IntersectTypeEquipment(t.RequiredEquipment, m.
            Equipment) {
                continue
            }
            act := int(t.RequiredEquipment)
            taskIdxByAction[act] = append(taskIdxByAction[act], ti)
90

```



```

95     }
    for a := range taskIdxByAction {
        availableActions = append(availableActions, a)
    }
    if len(availableActions) == 0 {
100        continue
    }

    // Инициализация Q для машины
    s.mu.Lock()
105    s.ensureQ(m.ID)
    s.mu.Unlock()

    // epsilon-greedy выбор действия (equipment)
    var chosenAction int
110    if rng.Float64() < s.Epsilon {
        // случайный выбор
        chosenAction = availableActions[rng.Intn(len(availableActions))]
    } else {
        // выбрать по Q
115        s.mu.Lock()
        ca, _ := s.argmaxQ(m.ID, availableActions)
        s.mu.Unlock()
        chosenAction = ca
    }

120    // в действии: выбрать конкретную задачу этого типа, например
    самую дешёвую"" по( energy cost)
    candidates := taskIdxByAction[chosenAction]
    if len(candidates) == 0 {
125        continue
    }

    // выбрать задачу с наименьшей energy_cost энергосбережение(
    мотивационно)
    bestTi := candidates[0]
    minCost := tasks[bestTi].EnergyCost
    for _, ti := range candidates {
130        if tasks[ti].EnergyCost < minCost {
            minCost = tasks[ti].EnergyCost
            bestTi = ti
        }
    }

135    // Выполнение: машина вносит всю свою энергию или( столько,
    сколько нужно)
    task := &tasks[bestTi]
    if m.Energy <= 0 || task.EnergyCost <= 0 {
140        continue
    }
    delta := m.Energy
    if delta > task.EnergyCost {
        delta = task.EnergyCost
    }

```

```

    }
145 // расходы и вклад делаем в основном потоке – безопасно
    m.Energy -= delta
    task.EnergyCost -= delta
    anyProgress = anyProgress || delta > 0

150 // Награда: положительная за завершение задачи; отрицательная за
    расход энергии
    var reward float64
    if task.EnergyCost <= 0 {
        // задача завершена
        task.EnergyCost = 0
155 _, existed := doneSet[task.ID]
        if !existed {
            doneSet[task.ID] = struct{}{}
            doneList = append(doneList, task.ID)
        }
160 // премия за завершение; масштабируем по размеру задачи
        reward = 10.0 + 0.01*float64(delta)
    } else {
        // частичное выполнение даёт слабую награду
        reward = 0.01 * float64(delta)
165 }
    // штраф за расход энергии чтобы( стимулировать экономию)
    reward -= 0.001 * float64(delta)

    // Q-update:  $Q(s,a) \leftarrow Q + lr*(reward + \gamma * \max_{a'} Q(s',a') - Q)$ 
170 // В данной простой модели состояние – остаток энергии машины
    дискретизируем( в бакеты)
    // Но для простоты используем одношаговый TD: следующая Q max для
    доступных действий
    var maxNext float64
    s.mu.Lock()
    // ensure entries exist for actions
175 for _, a := range availableActions {
        if _, ok := s.Q[m.ID][a]; !ok {
            s.Q[m.ID][a] = 0.0
        }
        if s.Q[m.ID][a] > maxNext {
180 maxNext = s.Q[m.ID][a]
        }
    }
    oldQ := s.Q[m.ID][chosenAction]
    newQ := oldQ + s.LR*(reward+s.Gamma*maxNext-oldQ)
185 s.Q[m.ID][chosenAction] = newQ
    s.mu.Unlock()
} // конец по машинам

// уменьшение epsilon мягкое()
190 s.mu.Lock()
if s.Epsilon > s.MinEps {

```

```

        s.Epsilon *= s.Decay
        if s.Epsilon < s.MinEps {
            s.Epsilon = s.MinEps
195     }
    }
    s.mu.Unlock()

    if !anyProgress {
200     break
    }
} // конец итераций

// итоговые метрики
205 finalTotalEnergy := sumEnergy()
energyUsed := initialTotalEnergy - finalTotalEnergy

stepMetrics := StepMetrics{
    EnergyUsed:    energyUsed,
210    TasksDone:    doneList,
    LenTasksDone:  len(doneList),
    LenNotExecTasks: 0,
    RealTime:      time.Since(start).String(),
}
215 for i := range tasks {
    if tasks[i].EnergyCost > 0 {
        stepMetrics.LenNotExecTasks++
    }
}
220

// безопасная запись в report
return stepMetrics
}

225 // helper: ensure Q entry exists
func (s *RLStrategy) ensureQ(machineID string) {
    if _, ok := s.Q[machineID]; !ok {
        s.Q[machineID] = make(map[int]float64)
    }
230 }

// argmax over equipment actions for given machine state (available actions)
func (s *RLStrategy) argMaxQ(machineID string, available []int) (
    bestAction int, bestVal float64) {
    s.ensureQ(machineID)
235 bestVal = math.Inf(-1)
    bestAction = -1
    for _, a := range available {
        v := s.Q[machineID][a]
        if v > bestVal || bestAction == -1 {
240     bestVal = v
            bestAction = a
        }
    }
}

```

```
    }  
    }  
    return  
245 }
```

A.13 Стратегия простого прямого перебора

Листинг 13: файл direct.go

```
package strategies

import (
    "time"
5
    "github.com/dreddsa5dies/phd/Dissertation/code/src/models"
)

// стратегия прямого перебора
10 type DirectSearch struct{}

func (s *DirectSearch) String() string {
    return "Прямого перебора"
}
15

// Стратегия заключается в последовательном проходе
// по машинам и задачам и их исполнения
func (s *DirectSearch) Run(machines []models.Machine, tasks []models.
    Task) StepMetrics {
    tNow := time.Now()
20

    // Инициализация метрик шага
    stepMetrics := StepMetrics{}

    // Флаг: смогла ли какаято- машина внести вклад на итерации
25 var progress bool

    // Внутренний цикл итераций – пока есть прогресс
    for {
        progress = false
30

        // Каждая машина пытается внести вклад в( порядке списка)
        for i := range machines {
            // Пропускаем, если машина без энергии
            if machines[i].Energy <= 0 {
35                continue
            }

            // Перебираем задачи в порядке следования
            for j := range tasks {
40                // Пропускаем выполненную задачу
                if tasks[j].EnergyCost <= 0 {
                    continue
                }

                // Проверка оборудования
45                if !models.IntersectTypeEquipment(tasks[j].RequiredEquipment,
                    machines[i].Equipment) {
```

```

        continue
    }

50    // Машина вносит вклад: minсвоей( энергии, остатка задачи)
    contribution := machines[i].Energy
    if contribution > tasks[j].EnergyCost {
        contribution = tasks[j].EnergyCost
    }

55    // Вносим вклад
    tasks[j].EnergyCost -= contribution
    machines[i].Energy -= contribution
    stepMetrics.EnergyUsed += contribution
    progress = true // был прогресс

    // Назначаем задачу как выполняемую для( отслеживания)
    machines[i].AssignedTask = &tasks[j]

65    // Если задача выполнена – увеличиваем счётчик
    if tasks[j].EnergyCost <= 0 {
        stepMetrics.LenTasksDone++
        stepMetrics.TasksDone = append(stepMetrics.TasksDone, tasks[j
].ID)
    }

70    // После вклада – машина может продолжить в следующей итерации,
    // но на этой итерации она работает только с первой подходящей
    задачей
    // по( принципу прямого” перебора” – одна попытка за итерацию)
    break
75    }
    }

    // Если ни одна машина не смогла внести вклад – выходим
    if !progress {
80        break
    }
}

// Подсчёт оставшихся невыполненных задач
85    for _, task := range tasks {
        if task.EnergyCost > 0 {
            stepMetrics.LenNotExecTasks++
        }
    }

90    stepMetrics.RealTime = time.Since(tNow).String()

    // возврат метрик
    return stepMetrics
95 }

```

A.14 Анализ, генерация графиков и таблиц

Листинг 14: файл analisys/main.go

```
package main

import (
    "encoding/csv"
    "encoding/json"
    "fmt"
    "log"
    "os"
    "sort"
    "strconv"
    "strings"

    "github.com/dreddsa5dies/phd/Dissertation/code/src/strategies"
    "gonum.org/v1/plot"
    "gonum.org/v1/plot/font"
    "gonum.org/v1/plot/plotter"
    "gonum.org/v1/plot/plotutil"
    "gonum.org/v1/plot/vg"
)

type (
    // ключ – имя стратегии
    ScenarioData map[string]strategies.Metrics

    // ключ – сценарий: "equal", "max", "min"
    AllData map[string]ScenarioData
)

var (
    bigFontSize    = 22
    largeFontSize  = 18
    meanFontSize   = 16
)

// Парсинг времени в микросекунды безопасно( с UTF-8)
func parseTime(s string) float64 {
    s = strings.TrimSpace(s)
    if s == "" {
        return 0
    }

    var unit string
    var valueStr string

    // Используем TrimSuffix для безопасного удаления суффиксов
    switch {
    case strings.HasSuffix(s, "ns"):
        unit = "ns"
```

```

50     valueStr = strings.TrimSpace(strings.TrimSuffix(s, "ns"))
    case strings.HasSuffix(s, "µs"): // Unicode символ- μ (U+00B5)
        unit = "µs"
        valueStr = strings.TrimSpace(strings.TrimSuffix(s, "µs"))
    case strings.HasSuffix(s, "us"): // fallback: если вдруг пришло как "
        us"
        unit = "µs"
55     valueStr = strings.TrimSpace(strings.TrimSuffix(s, "us"))
    case strings.HasSuffix(s, "ms"):
        unit = "ms"
        valueStr = strings.TrimSpace(strings.TrimSuffix(s, "ms"))
    case strings.HasSuffix(s, "s"):
60     unit = "s"
        valueStr = strings.TrimSpace(strings.TrimSuffix(s, "s"))
    default:
        unit = "µs"
        valueStr = s
65 }

    if valueStr == "" {
        return 0
    }

70     f, err := strconv.ParseFloat(valueStr, 64)
    if err != nil {
        return 0
    }

75     // Конвертируем всё в микросекунды (µs)
    switch unit {
    case "ns":
        return f / 1000.0 // нано → микро
80     case "µs", "us":
        return f
    case "ms":
        return f * 1000.0 // милли → микро
    case "s":
85     return f * 1_000_000.0 // секунды → микро
    default:
        return f
    }
}

90 // Получение количества выполненных задач на шаге
func getDoneCount(metrics []strategies.StepMetrics, step int) int {
    if step >= len(metrics) || step < 0 {
        return 0
95     }
    m := metrics[step]
    if len(m.TasksDone) > 0 {
        return len(m.TasksDone)
    }
}

```



```

100   return m.LenTasksDone
    }

    // Медиана
    func median(values []float64) float64 {
105     if len(values) == 0 {
        return 0
    }
    sort.Float64s(values)
    n := len(values)
110     if n%2 == 1 {
        return values[n/2]
    }
    return (values[n/2-1] + values[n/2]) / 2
    }
115

    // Сбор агрегированных результатов
    type ResultRow struct {
        Scenario      string
        Strategy       string
120     DoneTasks     int
        EnergyUsed     float64
        TotalEnergy    float64
        TotalTimeUS    float64
        PercentDone    float64
125     AvgEnergyPerTask float64
    }

    func collectAllResults(data AllData) []ResultRow {
        var results []ResultRow
130     scenarios := []string{"equal", "max", "min"}
        for _, scenario := range scenarios {
            scenarioData, ok := data[scenario]
            if !ok {
                continue
            }
135         for strategy, metrics := range scenarioData {
            done := getDoneCount(metrics.StepMetric, len(metrics.StepMetric)
            -1)
            totalTime := parseTime(metrics.TotalTime)
            energyUsed := 0.0
140         if len(metrics.StepMetric) > 0 {
            energyUsed = metrics.StepMetric[len(metrics.StepMetric)-1].
            EnergyUsed
        }
        percentDone := 0.0
        if metrics.TotalTasks > 0 {
145         percentDone = float64(done) / float64(metrics.TotalTasks) * 100
        }
        avgEnergyPerTask := 0.0
        if done > 0 {
            avgEnergyPerTask = energyUsed / float64(done)
        }
    }

```

```

150     }
        results = append(results, ResultRow{
            Scenario:      scenario,
            Strategy:      strategy,
            DoneTasks:     done,
155            EnergyUsed:   energyUsed,
            TotalEnergy:   metrics.TotalEnergyMachines,
            TotalTimeUS:   totalTime,
            PercentDone:   percentDone,
            AvgEnergyPerTask: avgEnergyPerTask,
160        })
    }
}
return results
}

165 // Сохранение CSV
func saveSummaryCSV(results []ResultRow) {
    f, err := os.Create("summary_results.csv")
    if err != nil {
170        log.Fatalf("Ошибка создания CSV: %v", err)
    }
    defer f.Close()

    w := csv.NewWriter(f)
175    defer w.Flush()

    headers := []string{"Сценарий", "Стратегия", "Выполнено задач", "
        Использовано энергии", "Всего энергии", "% выполнения", "Ср. энергия
        на задачу", "Общее время ( $\mu$ s)"}
    if err := w.Write(headers); err != nil {
        log.Fatalf("Ошибка записи заголовка: %v", err)
180    }

    for _, r := range results {
        record := []string{
            r.Scenario,
185            r.Strategy,
            strconv.Itoa(r.DoneTasks),
            fmt.Sprintf("%.2f", r.EnergyUsed),
            fmt.Sprintf("%.2f", r.TotalEnergy),
            fmt.Sprintf("%.1f%%", r.PercentDone),
190            fmt.Sprintf("%.2f", r.AvgEnergyPerTask),
            fmt.Sprintf("%.3f", r.TotalTimeUS),
        }
        if err := w.Write(record); err != nil {
            log.Printf("Ошибка записи строки: %v", err)
195        }
    }

    fmt.Println("CSV сохранён: summary_results.csv")
}

```

```

200 // Группированный barчарт- по полю
func plotGroupedBarChart(results []ResultRow, field, _, ylabel string,
    getValue func(ResultRow) float64) {
    scenarios := []string{"equal", "max", "min"}
    strategiesMap := make(map[string]bool)
205 for _, r := range results {
    strategiesMap[r.Strategy] = true
}
var strategies []string
for s := range strategiesMap {
210 strategies = append(strategies, s)
}
sort.Strings(strategies)

data := make(map[string]map[string]float64)
215 for _, sc := range scenarios {
    data[sc] = make(map[string]float64)
}
for _, r := range results {
    data[r.Scenario][r.Strategy] = getValue(r)
220 }

p := plot.New()

// Настройка увеличенных шрифтов
225 p.Y.Label.Text = ylabel
p.Y.Label.TextStyle.Font.Size = font.Length(largeFontSize)
p.X.Tick.Label.Rotation = 0.5
p.X.Tick.Label.Font.Size = font.Length(meanFontSize)
p.Y.Tick.Label.Font.Size = font.Length(meanFontSize)
230 p.X.Padding = 30

// Увеличенная ширина столбцов
width := vg.Points(40)
// Уменьшенный отступ между группами
235 sep := vg.Points(10)

barCharts := make([]plot.Plotter, 0)

for i, strategy := range strategies {
240 values := make(plotter.Values, len(scenarios))
    for j, sc := range scenarios {
        values[j] = data[sc][strategy]
    }
    bars, err := plotter.NewBarChart(values, width)
245 if err != nil {
        continue
    }
    color := plotutil.Color(i)
    bars.Color = color
250 // Плотное размещение: меньше смещение

```

```

bars.Offset = vg.Length(i)*(width+vg.Points(5)) - sep/2
barCharts = append(barCharts, bars)

// Добавляем в легенду с цветом
255 p.Legend.Add(strategy, bars)
}

for _, bc := range barCharts {
    p.Add(bc)
260 }
p.NominalX("equal", "max", "min")
p.Legend.Top = true
p.Legend.TextStyle.Font.Size = font.Length(bigFontSize)

265 if err := p.Save(1000, 600, "bar_"+strings.ReplaceAll(field, " ", "_")
    +".png"); err != nil {
    log.Printf("Ошибка сохранения bar_%s.png: %v", field, err)
} else {
    fmt.Printf("График сохранён: bar_%s.png\n", strings.ReplaceAll(field
        , " ", "_"))
}
270 }

// Barчарты-: среднее и медиана выполненных задач по шагам
func plotMeanMedianBarCharts(data AllData) {
    scenarios := []string{"equal", "max", "min"}
275 strategiesMap := make(map[string]bool)

    type Stats struct {
        Mean    float64
        Median float64
280 }
    stats := make(map[string]map[string]Stats)

    for _, scenario := range scenarios {
        scenarioData, ok := data[scenario]
285 if !ok {
            continue
        }
        stats[scenario] = make(map[string]Stats)
        for strategy, metrics := range scenarioData {
290 var values []float64
            for step := 0; step < len(metrics.StepMetric); step++ {
                done := float64(getDoneCount(metrics.StepMetric, step))
                values = append(values, done)
            }
295 if len(values) == 0 {
                values = []float64{0}
            }

            var sum float64
300 for _, v := range values {

```

```

        sum += v
    }
    mean := sum / float64(len(values))
    medianVal := median(values)
305
    strategiesMap[strategy] = true
    stats[scenario][strategy] = Stats{Mean: mean, Median: medianVal}
}
}
310
var strategies []string
for s := range strategiesMap {
    strategies = append(strategies, s)
}
315
sort.Strings(strategies)

width := vg.Points(40)
sep := vg.Points(10)

320 // Среднее
{
    p := plot.New()

    // Настройка увеличенных шрифтов
325 p.Y.Label.Text = "Среднее по( шагам)"
    p.Y.Label.TextStyle.Font.Size = font.Length(largeFontSize)
    p.X.Tick.Label.Rotation = 0.5
    p.X.Tick.Label.Font.Size = font.Length(meanFontSize)
    p.Y.Tick.Label.Font.Size = font.Length(meanFontSize)
330 p.X.Padding = 30

    barCharts := make([]plot.Plotter, 0)

    for i, strategy := range strategies {
335         values := make(plotter.Values, len(scenarios))
        for j, sc := range scenarios {
            values[j] = stats[sc][strategy].Mean
        }
        bars, err := plotter.NewBarChart(values, width)
340         if err != nil {
            continue
        }
        color := plotutil.Color(i)
        bars.Color = color
345         bars.Offset = vg.Length(i)*(width+vg.Points(5)) - sep/2
        barCharts = append(barCharts, bars)

        p.Legend.Add(strategy, bars)
    }
350
    for _, bc := range barCharts {
        p.Add(bc)
    }
}

```

```

    }
    p.NominalX("equal", "max", "min")
355 p.Legend.Top = true
    p.Legend.TextStyle.Font.Size = font.Length(bigFontSize)

    if err := p.Save(1000, 600, "bar_done_tasks_mean.png"); err != nil {
        log.Printf("Ошибка сохранения bar_done_tasks_mean.png: %v", err)
360 } else {
        fmt.Println("График сохранён: bar_done_tasks_mean.png")
    }
}

365 // Медиана
{
    p := plot.New()

    // Настройка увеличенных шрифтов
370 p.Y.Label.Text = "Медиана по( шагам)"
    p.Y.Label.TextStyle.Font.Size = font.Length(largeFontSize)
    p.X.Tick.Label.Rotation = 0.5
    p.X.Tick.Label.Font.Size = font.Length(meanFontSize)
    p.Y.Tick.Label.Font.Size = font.Length(meanFontSize)
375 p.X.Padding = 30

    barCharts := make([]plot.Plotter, 0)

    for i, strategy := range strategies {
380 values := make(plotter.Values, len(scenarios))
        for j, sc := range scenarios {
            values[j] = stats[sc][strategy].Median
        }
        bars, err := plotter.NewBarChart(values, width)
385 if err != nil {
            continue
        }
        color := plotutil.Color(i)
        bars.Color = color
390 bars.Offset = vg.Length(i)*(width+vg.Points(5)) - sep/2
        barCharts = append(barCharts, bars)

        p.Legend.Add(strategy, bars)
    }

395 for _, bc := range barCharts {
        p.Add(bc)
    }
    p.NominalX("equal", "max", "min")
400 p.Legend.Top = true
    p.Legend.TextStyle.Font.Size = font.Length(bigFontSize)

    if err := p.Save(1000, 600, "bar_done_tasks_median.png"); err != nil
    {

```

```

    log.Printf("Ошибка сохранения bar_done_tasks_median.png: %v", err)
405 } else {
    fmt.Println("График сохранён: bar_done_tasks_median.png")
    }
}
}
410
// Основная функция
func main() {
    // Чтение файла
    b, err := os.ReadFile("report.json")
415 if err != nil {
    log.Fatalf("Ошибка чтения report.json: %v", err)
    }

    var all AllData
420 if err := json.Unmarshal(b, &all); err != nil {
    log.Fatalf("Ошибка парсинга JSON: %v", err)
    }

    // Сбор данных
425 results := collectAllResults(all)

    // Сохранение CSV
    saveSummaryCSV(results)

430 // Группированные bar-чарты-
    plotGroupedBarChart(results, "Done Tasks", "Выполненные задачи", "
        Количество", func(r ResultRow) float64 {
        return float64(r.DoneTasks)
    })

435 plotGroupedBarChart(results, "Energy Used", "Использованная энергия",
    "Энергия", func(r ResultRow) float64 {
        return r.EnergyUsed
    })

    plotGroupedBarChart(results, "Total Time", "Общее время выполнения", "
        Время (µs)", func(r ResultRow) float64 {
440 return r.TotalTimeUS
    })

    // Среднее и медиана по шагам
    plotMeanMedianBarCharts(all)
445
    fmt.Println("Анализ завершён. Файлы: summary_results.csv, bar_*.png")
}

```